

3-Mavzu: Pythonda tarmoq dasturlashga kirish.
4-mashg‘ulot. TCP klient-server dasturini testlash.

O‘quv savollari:

1. TCP client-server dasturi yordamida ma’lumot almashish.
2. TCP ulanishini o‘rnatish va testlash.
3. TCP client va serverdagi xatoliklarni aniqlash va bartaraf etish.

Maqsad:

Python dasturlash tilida **TCP protokoli** asosida klient-server dasturini yaratish, u orqali ma’lumot almashishni amalga oshirish, ulanishni testlash va xatoliklarni aniqlab bartaraf etish.

1. TCP Client-server dasturi yordamida ma’lumot almashish

TCP Protokoli haqida qisqacha

TCP (Transmission Control Protocol) – bu tarmoq orqali **ishonchli ma’lumot uzatishni ta’minlovchi protokol** bo‘lib, u ma’lumotlarni ketma-ket paketlar holida yuboradi va qabul qiladi. TCP har bir paket yetkazilganini tasdiqlaydi va yo‘qolgan paketlarni qayta yuboradi.

Socket moduli

Python’da tarmoq dasturlash uchun `socket` moduli ishlatiladi. U **klient** va **server** o‘rtasida ma’lumot almashishga imkon beradi.

Asosiy metodlar:

Metod	Tavsif
<code>socket.socket()</code>	Soket obyektini yaratadi
<code>bind()</code>	Serverni IP manzil va portga bog‘laydi
<code>listen()</code>	Ulanishlarni kutishga o‘rnatadi
<code>accept()</code>	Mijozdan ulanishni qabul qiladi
<code>connect()</code>	Serverga ulanishni amalga oshiradi
<code>send()</code> / <code>sendall()</code>	Ma’lumot yuboradi
<code>recv()</code>	Ma’lumot qabul qiladi
<code>close()</code>	Ulanishni yopadi

Server kodi

```
import socket

# Server soketini yaratish
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# IP va portni bog‘lash
server_ip = "192.168.1.10"
server_port = 5000
server_socket.bind((server_ip, server_port))
server_socket.listen(5)
```

```

print(f"Server {server_ip}:{server_port} manzilida ulanishni kutmoqda...")

# Ulanishni qabul qilish
conn, addr = server_socket.accept()
print(f"Yangi mijoz ulandi: {addr}")

# Mijozdan xabar olish
data = conn.recv(1024)
print(f"Mijozdan olingan ma'lumot: {data.decode('utf-8')}")

# Javob yuborish
conn.sendall("Salom, klient! Men serverman.".encode('utf-8'))

# Ulanishni yopish
conn.close()

```

Klient kodi

```

import socket

# Klient soketini yaratish
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# Serverga ulanish
server_ip = "192.168.1.10"
server_port = 5000
client_socket.connect((server_ip, server_port))

# Serverga xabar yuborish
message = "Salom, server!"
client_socket.sendall(message.encode('utf-8'))

# Serverdan javob olish
response = client_socket.recv(1024)
print(f"Serverdan javob: {response.decode('utf-8')}")

# Soketni yopish
client_socket.close()

```

2. TCP Ulanishini O'rnatish va Testlash

Ulanish bosqichlari

1. **Server** `bind()` va `listen()` yordamida ulanishlarni kutadi.
2. **Klient** `connect()` yordamida serverga ulanadi.
3. **Server** `accept()` orqali yangi ulanishni qabul qiladi.
4. **Ma'lumot almashish** `send()` va `recv()` metodlari yordamida amalga oshadi.
5. **Ulanishni yakunlash** `close()` orqali bajariladi.

Testlash jarayoni

1-qadam: Server dasturini ishga tushiring

```
python server.py
```

2-qadam: Klient dasturini ishga tushiring

```
python client.py
```

3-qadam: Natijani tekshiring

- Klientdan: “Salom, server!” yuboriladi
- Serverdan: “Salom, klient! Men serverman.” javobi olinadi

Natija ekranda quyidagicha ko‘rinadi:

```
Server 192.168.1.10:5000 manzilida ulanishni kutmoqda...
Yangi mijoz ulandi: ('192.168.1.11', 50532)
Mijozdan olingan ma'lumot: Salom, server!
```

3. TCP Client va serverdagi xatoliklarni aniqlash va bartaraf etish

Xatoliklarni qayta ishlash (try-except)

Tarmoq ulanishlarida xatoliklar (masalan, server ishlamayapti, port band, tarmoq uzilgan) tez-tez uchraydi. Shu sababli try-except bloklari ishlatiladi.

```
import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

try:
    client_socket.connect(("192.168.1.10", 5000))
    print("Serverga muvaffaqiyatli ulandik!")
except socket.error as e:
    print(f"Ulanishda xatolik: {e}")
```

Timeout belgilash

Ulanish yoki javobni kutish uchun vaqt chegarasini o‘rnatish:

```
client_socket.settimeout(10) # 10 soniya
try:
    client_socket.connect(("192.168.1.10", 5000))
except socket.timeout:
    print("Ulanish vaqti tugadi!")
```

Qayta ulanish (reconnect)

Agar dastur uzilib qolsa, quyidagi usul yordamida qayta ulanadi:

```
def reconnect():
    try:
        client_socket.connect(("192.168.1.10", 5000))
        print("Serverga qayta ulanish amalga oshirildi.")
    except socket.error as e:
        print(f"Ulanishda xatolik: {e}")
        reconnect()
```

Ulanishni to'g'ri yopish

Resurslarni to'g'ri boshqarish uchun har ikkala tomon socketni yopishi kerak.

```
conn.close()          # serverda
client_socket.close() # klientda
```

Xulosa

- TCP yordamida **ishonchli tarmoq ulanishi** o'rnatish va ma'lumot almashish mumkin.
- `socket` moduli asosida client-server modeli yaratiladi.
- `try-except` bloklari yordamida xatoliklar boshqariladi.
- `timeout`, `reconnect`, va `close` metodlari ulanish sifatini boshqarish uchun muhim.